1.1 Modes d’adressage

Que remarquez-vous concernant le contenu des différents registres ?

> Plusieurs registres peuvent contenir la même donnée lorsqu'ils accèdent à la même adresse mémoire, même si le mode d'adressage utilisé est différent.

Que remarquez-vous concernant la mémoire ?

Instruction	Code machine (Hexa)	Adresse logique	Adresse physique	Registre ou cases mémoires modifiées	Commentaire (opération + mode d'adressage)
org 100h					Adresse de départ du code/des données
mov ax, 15h	B81500	0700:0100	07100	AX , IP	Charger ax avec 15h Adressage immédiat de registre
mov bx,2502h	BB0225	0700:0103	07103	BX , IP	Charger bx avec 2502h Adressage immédiat de registre
mov w.[0300h],ax	A30003	0700:0106	07106	[0300h] , IP	Stocke le contenu de ax à l'adresse mémoire DS :0300h => Adressage immédiat de la mémoire
mov w.[03002h],bx	891E0203	0700:0109	07109	[0302h] , [0303h] , IP	Stocke le contenu de bx à l'adresse mémoire DS :0302h => Adressage immédiat de la mémoire
mov [03004h],35h	C606040335	0700:010D	0710D	[0304h] , IP	Stocke l'octet 35h à l'adresse DS :0304h Adressage immédiat de la mémoire
mov bx,0300h	BB0003	0700:0112	07112	BX , IP	Charger bx avec 0300h (pointeur vers la zone mémoire) => Adressage immédiat de registre
mov cx,[bx]	8B0F	0700:0115	07115	CX , IP	Charger cx avec le contenu de la zone mémoire DS :bx (DS :0300h) => Adressage basé
mov dx,[bx+2]	8B5702	0700:0117	07117	DX , IP	Charge dx avec DS :bx+2 (DS :0302) Adressage basé
mov ax,[bx]+4	8B4704	0700:011A	0711A	AX , IP	Charger ax avec (le contenu mémoire DS :0300h)+ 4 => Adressage basé
mov si,bx	8BF3	0700:011D	0711D	SI , IP	Charger le registre SI avec 0300h Adressage par registre
mov di,[si]	8B3C	0700:011F	0711F	DI , IP	Charger DI avec le contenu du mot à DS :0300h => Adressage indexé
ret	C3	0700:0121	07121	SP, IP	Fin du programme .COM

1.2.1 Opérations arithmétiques

Visualiser le contenu du registre d'états (flags)

Les registres OF et CF sont égaux à 1 car le résultat de la multiplication dépasse la capacité du registre AX, ce qui nécessite l'utilisation du registre DX pour stocker les bits de poids fort.

Conclure

Le mode single run effectue toutes les instructions automatiquement les unes après les autres**,** sans s'arrêter après chaque instruction.

```
; fixer le registre de segment de donnees "DS" sur 100h pour
; pouvoir sauvegarder les resultats dans la memoire
; a partir de l'adresse logique 100h : 300h

mov ax, 100h
mov ds, ax

mov ax, var1
add ax, var2

mov [0300h], ax

mov ax, var1
sub ax, var2

mov [0300h]+2, ax

mov ax, var1
mul var2

mov [0300h]+4, ax
mov [0300h]+6, dx

mov ax, var1
mov bx, var2

; on met dx a zero pour eviter les erreurs
; car la DIV utilise DX comme la "suite" de AX pour former
; un seul grand nombre de 32 bits

mov dx, 0

div bx

mov [0300h]+8, ax
mov [0300h]+10, dx

ret

; premier facon de declarer les variables

var1 DW 1525h
var2 DW 1000h
```

```
; au lieu d'utiliser

; mov ax, 100h
; mov ds, ax

; on peut ecrire org 0h

org 0h

; deuxieme facon de declarer les variables

jmp start

var1 DW 1525h
var2 DW 1000h

start:

mov ax, var1
add ax, var2

mov [0300h], ax

mov ax, var1
sub ax, var2

mov [0300h]+2, ax

mov ax, var1
mul var2

mov [0300h]+4, ax
mov [0300h]+6, dx

mov ax, var1
mov bx, var2

mov dx, 0

div bx

mov [0300h]+8, ax
mov [0300h]+10, dx

ret
```

```
; il ne faut pas utiliser org lorsqu'on utitlise les segments
; org 0h

; troisieme facon de declarer les variables
; en utilisant le data segement on ne peut que commencer
; le segment de donnees a partir de l'adresse 0710h : 100h
; (ou n'importe quelle adresse situee dans le registre ax au debut)

data segment

var1 DW 1525h
var2 DW 1000h

data ends

code segment

    assume ds:data, cs:code

    start:

        mov ax, data
        mov ds, ax

        mov ax, var1
        add ax, var2

        mov [0300h], ax

        mov ax, var1
        sub ax, var2

        mov [0300h]+2, ax

        mov ax, var1
        mul var2

        mov [0300h]+4, ax
        mov [0300h]+6, dx

        mov ax, var1
        mov bx, var2

        mov dx, 0

        div bx

        mov [0300h]+8, ax
        mov [0300h]+10, dx

        hlt

    code ends

end start
```

1.2.2 Opérations logiques

Visualiser le contenu du registre d'états (flags)

Les drapeaux ZF et PF sont égaux à 1 a cause de l'operation AND

Les drapeaux SF et PF sont égaux à 1 a cause de l'operation OR

Les drapeaux SF et PF sont égaux à 1 a cause de l'operation XOR

Les drapeaux AF et CF sont égaux à 1 a cause de l'operation NEG

Conclure

Le mode single run effectue toutes les instructions automatiquement les unes après les autres**,** sans s'arrêter après chaque instruction.

```
org 100h

mov al, var1
and al, var2

mov result1, al

mov al, var1
or al, var2

mov result2, al

mov al, var1
xor al, var2

mov result3, al

mov al, var1
not al

mov result4, al

mov al, var1
neg al

mov result5, al

ret

var1 db 11001100b
var2 db 00110011b

; donnees initials aleatoire

result1 db 10101010b
result2 db 10101010b
result3 db 10101010b
result4 db 10101010b
result5 db 10101010b
```

1.3 Boucles et instructions de branchement : Opérations sur les tableaux

Quelles sont les adresses logique et physique des variables min et max.

Min : 07109 , 0710 : 0009

Max : 07108 , 0710 : 0008

Visualiser l'évolution du contenu de ces deux variables. Conclure

À chaque itération sur la table les variables max et min sont mises à jour si nécessaire, à la fin de la boucle, max contient la plus grande valeur de la table et min la plus petite.

```
data segment

    table db 15, 20, 10, 11, 2, 35, 60, 30
    max db ?
    min db ?
    taille db 8

data ends

code segment

    assume ds : data, cs : code

    start:

        mov ax, data
        mov ds, ax

        ; on met l'adresse de la variable table dans si

        lea si, table

        ; on met la donnee (table) dont l'offset est contenu dans si

        mov al, [si]

        ; les variables max et min et taille sont des octets de 8 bits
        ; donc on doit utiliser 8 bits du registre ax (al) et 8 bits du registre cx (cl)

        mov max, al
        mov min, al

        mov cl, taille
        dec cl

        ; le si s'incremente pour acceder au deuxieme element de la table

        inc si

        boucle:
            mov al, [si]
            cmp al, max

            ; si le nombre actuel est <= au max verifie s'il peut etre un nouveau min
            jbe minimum

            ; sinon c'est le nouveau max
            mov max, al

        minimum:
            cmp al, min

            ; si le nombre actuel est >= au min sauter vers le nombre suivant
            jae suivant

            ; sinon c'est le nouveau min
            mov min, al

        suivant:
            inc si
            dec cl

            ; si cl n'est pas nul refaire la boucle
            jnz boucle

        hlt

    code ends

end start
```

```
data segment

    table  dw 15, 20, 10, 11, 2, 35, 60, 30
    max    dw ?
    min    dw ?
    taille dw 8

data ends

code segment

    assume ds : data, cs : code

    start:

        mov ax, data
        mov ds, ax

        ; on met l'adresse de la variable table dans si

        lea si, table

        ; on met la donnee (table) dont l'offset est contenu dans si

        mov ax, [si]

        mov max, ax
        mov min, ax

        mov cx, taille
        dec cx

        ; le si s'incremente pour acceder au deuxieme element de la table

        ; pour utiliser les 16 bits des registres ax et cx on doit
        ; incrementer le si 2 fois car chaque nombre prendra 2 lignes
        ; d'adresse de 8 bits

        inc si
        inc si

        boucle:
            mov ax, [si]
            cmp ax, max

            ; si le nombre actuel est <= au max verifie s'il peut etre un nouveau min
            jbe minimum

            ; sinon c'est le nouveau max
            mov max, ax

        minimum:
            cmp ax, min

            ; si le nombre actuel est >= au min sauter vers le nombre suivant
            jae suivant

            ; sinon c'est le nouveau min
            mov min, ax

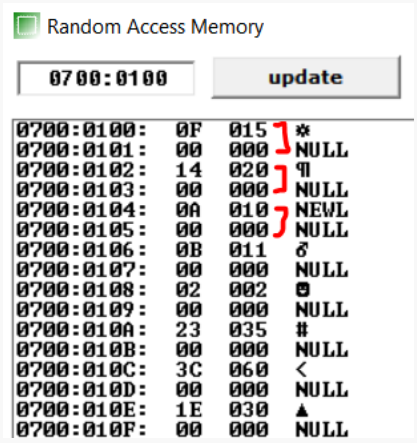
        suivant:
            inc si
            inc si
            dec cx

            ; si cx n'est pas nul refaire la boucle
            jnz boucle

        hlt

code ends

end start
```



1.4 Instructions de décalage et rotation

Executer les codes que vous avez developpé. Conclure

SHL et SHR déplacent les bits vers la gauche ou la droite et certains bits peuvent se perdre, alors que ROL et ROR font tourner les bits en cercle sans en perdre aucun.

```
org 100h

mov al, 20
shl al, 2

mov al, 20
shr al, 2

mov al, 11110000b
rol al, 4

mov al, 11110000b
ror al, 4

ret
```